

Module 02: Agents — The Agent Class and A–E Matrices

Learning Objectives

1. Construct a `GenerativeModel` by specifying A, B, C, D, and E matrices with correct shapes and normalization.
2. Initialize an `ActiveInferenceAgent` with a generative model, policies, and precision parameter γ .
3. Run the basic perception-action loop: `agent.step(observation) → action`.

Introduction

An Active Inference agent is defined by its **generative model** — a set of matrices that encode what the agent believes about how observations are generated (A), how states evolve (B), which outcomes are preferred (C), what the initial state is (D), and which policies are habitual (E). This module walks through each matrix and shows how to assemble them into a working agent.

Key Concepts

1. The A-Matrix: Likelihood

The A-matrix encodes $P(o \mid s)$ — the agent's belief about which observations arise from which states:

```

import numpy as np
from active_inference.agent import GenerativeModel

# 2 observations, 2 states
A = np.array([[0.9, 0.1],    # P(o=0 | s=0) = 0.9, P(o=0 | s=1) = 0.1
              [0.1, 0.9]])  # P(o=1 | s=0) = 0.1, P(o=1 | s=1) = 0.9

```

Validation rule: Each column of A must sum to 1.0 (it is a conditional distribution over observations given a state).

2. The B-Matrix: Transitions

The B-matrix encodes $P(s' \mid s, a)$ — how hidden states evolve under each action:

```

# 2 states, 2 actions
B = np.zeros((2, 2, 2))
B[:, :, 0] = np.eye(2)          # action 0: stay
B[:, :, 1] = np.array([[0, 1],
                        [1, 0]]) # action 1: swap

```

Shape: `(num_states, num_states, num_actions)`. Each slice `B[:, :, a]` is a column-stochastic transition matrix. A 2-D B (shape `(N, N)`) is treated as a single-action model.

3. The C-Vector: Preferences

The C-vector encodes the agent's **log-preferences** over observations. It answers: "Which observations does the agent want to experience?"

```

# Prefer observation 0 over observation 1
C = np.array([2.0, -2.0]) # log scale

```

C enters the Expected Free Energy (EFE) as the **risk** term: policies that lead to preferred observations have lower $G(\pi)$. A uniform C (all zeros) means the agent has no preferences and will be purely epistemic (information-seeking).

4. The D-Vector: Prior Over Initial States

The D-vector is $P(s_0)$ — the agent's prior belief about its starting state:

```
D = np.array([0.5, 0.5])    # uniform prior
```

Validation rule: D must sum to 1.0.

5. The E-Vector: Habit Prior

The optional E-vector encodes a **habit prior** over policies — $P(\pi)$ before considering EFE:

```
E = np.array([0.7, 0.3])    # prefer policy 0
```

When E is provided, the policy posterior becomes $q(\pi) = \sigma(-\gamma \cdot G(\pi) + \ln E(\pi))$. When E is `None`, a uniform prior is used.

6. Assembling the GenerativeModel

```
model = GenerativeModel(A=A, B=B, C=C, D=D, E=E)
print(model)    # GenerativeModel(obs=2, states=2, actions=2)
```

The constructor validates:

- A is 2-D with normalized columns
- B is 2-D or 3-D with normalized columns
- C has length `num_obs`
- D has length `num_states` and sums to 1
- E (if provided) sums to 1

7. The ActiveInferenceAgent

The agent wraps a generative model and adds inference machinery:

```
from active_inference.agent import ActiveInferenceAgent

agent = ActiveInferenceAgent(model, gamma=4.0)
print(agent)  # ActiveInferenceAgent( $\gamma$ =4.0, policies=2)
```

Key parameters:

- `gamma` (γ): precision of policy selection. Higher $\gamma \rightarrow$ more exploitative.
- `policies`: list of action sequences (default: one single-step policy per action).

8. The Perception-Action Loop

```
from active_inference.agent import DiscreteEnvironment

env = DiscreteEnvironment(A, B, initial_state=0)
agent = ActiveInferenceAgent(model, gamma=4.0)

obs = env.reset(initial_state=0)
for t in range(10):
    action = agent.step(obs)      # infer states  $\rightarrow$  infer policies  $\rightarrow$  select
    obs = env.step(action)        # environment transitions and emits observations
```

The `agent.step(obs)` method is a convenience that calls `infer_states(obs)`, `infer_policies()`, and `select_action()` in sequence.

Applications

- **T-Maze:** A classic benchmark with 4 states (center, left arm, right arm, cue location), 3 observations (center, reward, no-reward), and 3 actions (stay, go-left, go-right). The C-vector encodes preference for the reward observation.
- **Minimal 2-state model:** The simplest possible Active Inference agent that can be fully analyzed by hand — useful for pedagogical derivations.
- **Precision tuning:** Varying γ from 0.01 (random policy selection) to 100 (greedy) reveals the exploration–exploitation tradeoff.

Conclusion

The A–E matrices fully specify an Active Inference agent's generative model. With these matrices validated and an `ActiveInferenceAgent` instantiated, the agent can perceive, decide, and act. Module 03 will dive into the perception step — how beliefs are updated using the A-matrix and variational inference.